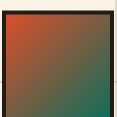




LLM COST ENGINEERING

You're paying full price for the same prompt, every single call.

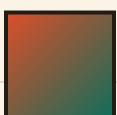
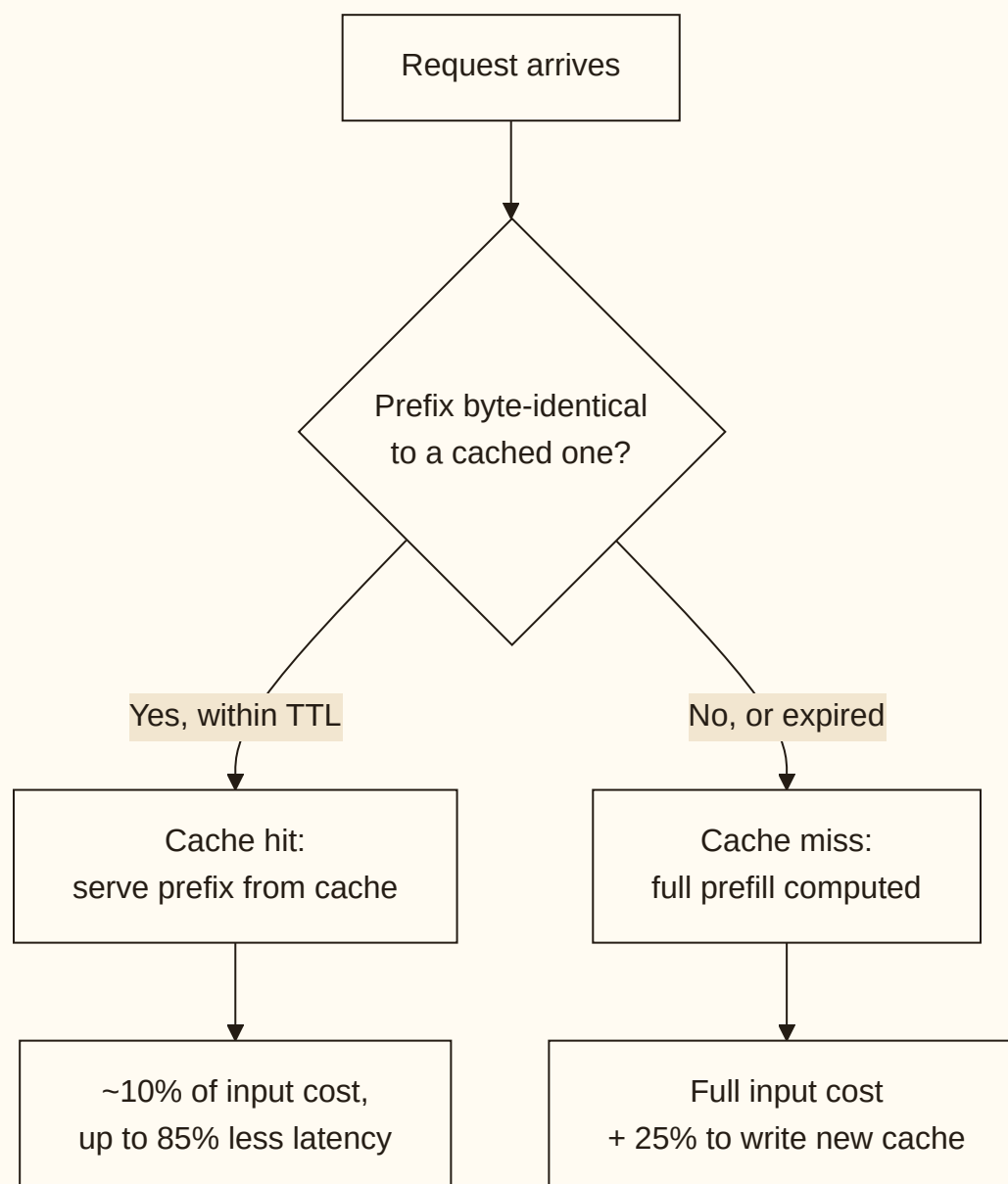
Prompt caching cuts input cost up to 90% and latency up to 85% — most teams never actually get a hit.



Sandesh Kale
GenAI Solutions Architect



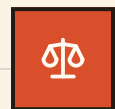
What Actually Happens On Each Call



Why Teams Never Actually Get A Hit



A timestamp, request ID, or user name gets interpolated into the SYSTEM prompt — the prefix is now different on every single call, guaranteed miss.



Dynamic content (the user's actual question) sits before the static content (docs, tool defs) instead of after it — nothing after the first byte-diff point can be cached.



The cache TTL (5 minutes by default) quietly expires between calls in a low-traffic path, so every request looks like the first one.



Nobody's actually checking `cache_read` vs `cache_creation` token counts — the cache can be missing 100% of the time and the bill looks "normal."





85%

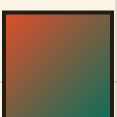
Lower latency on a cache hit

Same model, same prompt, same output length — the only difference is whether the long static prefix (system prompt, tool definitions, reference docs) was served from cache or recomputed from scratch.

Cache miss — full prefill



Cache hit — served from cache



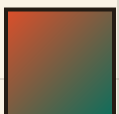
How To Actually Get The Hit

- 01 Order the prompt static-first: system prompt, tool definitions, reference docs, THEN the user's actual (dynamic) question last.

- 02 Keep the static prefix byte-identical across calls — no timestamps, no request IDs, no reordered JSON keys anywhere before the dynamic part.

- 03 Batch requests to land inside the TTL window instead of spacing them out past it, especially on a low-traffic endpoint.

- 04 Actually log `cache_read_input_tokens` vs `cache_creation_input_tokens` per call — a cache that silently never hits looks identical to a working one on every other metric.





Reorder your prompt before you reach for a smaller model.

A cache hit is a bigger lever than most model-downsizing decisions. Follow for the next cost-engineering breakdown.

